Docker

полная картина: как он устроен и как им пользоваться

Этот документ строит ментальную модель: сначала **что такое контейнер на самом деле** (спойлер: обычный процесс Linux с ограничениями), потом как вокруг этой идеи устроены образы, Dockerfile, данные, сети и Compose. Команды идут не отдельным списком «заучить», а как следствие понимания. В конце есть шпаргалка.

> **Главная мысль, на которой держится всё остальное**
> Docker не создаёт «маленьких компьютеров». Он просит ядро Linux запустить обычный процесс, но **показать ему урезанный мир**: свои файлы, свою сеть, свой список процессов и лимиты на ресурсы. Всё остальное в Docker — удобная упаковка, доставка и управление этим процессом.

Содержание

1	Какую проблему решает Docker
2	Контейнер — это процесс, а не виртуальная машина
3	Из чего ядро Linux собирает контейнер
4	Архитектура Docker: кто что делает
5	Образ: слои, имена, теги
6	Контейнер: жизненный цикл и главный процесс
7	Dockerfile: как собирается образ
8	Данные: тома и привязанные папки
9	Сети и порты
10	Реестры: где живут образы
11	Docker Compose: много контейнеров одним файлом
12	Безопасность: что важно знать
13	Шпаргалка команд
14	Итоговая картина и куда углубляться дальше

1. Какую проблему решает Docker

Любая программа зависит не только от своего кода, но и от **окружения**: версии языка (Python 3.12, Node 20), системных библиотек (libssl, libpq), настроек, переменных окружения, структуры папок. На твоём ноутбуке всё стоит и работает, на сервере — другая версия библиотеки, и программа падает. Классика: «у меня же работает».

Раньше это решали двумя способами: длинной инструкцией «установи вот это» (ломается при каждом обновлении) или виртуальной машиной (надёжно, но тяжело: целая ОС на каждое приложение).

Docker предлагает третий путь: **упаковать программу вместе со всем её окружением** в один неизменяемый пакет (образ) и запускать этот пакет одинаково на любой машине с Docker. При этом не нужна отдельная ОС на каждое приложение — все используют ядро хоста.

Что это даёт на практике

- **Воспроизводимость.** Один и тот же образ на ноутбуке, в автотестах и на сервере — значит одно и то же поведение.
- **Чистота системы.** Postgres 13 и Postgres 16 одновременно, разные версии Node — без конфликтов и мусора в системе. Удалил контейнер — следов нет.
- **Простая доставка.** Не «скопируй код и поставь зависимости», а «скачай образ и запусти».
- **Описание инфраструктуры текстом.** Dockerfile и compose.yaml лежат в git рядом с кодом: окружение становится частью проекта, а не устным знанием.
- **Изоляция.** Приложения не видят файлы и процессы друг друга (с оговорками, см. раздел 12).

> **Граница ответственности**
> Docker решает задачу **«как упаковать и запустить»**. Он не решает задачу «как управлять сотней серверов» — для этого есть оркестраторы (Kubernetes, Docker Swarm, Nomad). Они запускают те же самые образы, поэтому знание Docker — фундамент и для них.

2. Контейнер — это процесс, а не виртуальная машина

Это самый важный раздел. Если понять его, остальное складывается само.

Виртуальная машина эмулирует целый компьютер: у неё своё виртуальное «железо» и своё ядро ОС. Гипервизор делит физическое железо между такими машинами. Внутри ВМ может быть любая ОС — хоть Windows на Linux-сервере.

Контейнер ничего не эмулирует. Это обычный процесс на хосте, которому ядро Linux «надело шоры»: процесс видит только свои файлы, свою сеть и свои процессы, и ему выставлены лимиты на память и процессор. Ядро у всех контейнеров одно — ядро хоста.

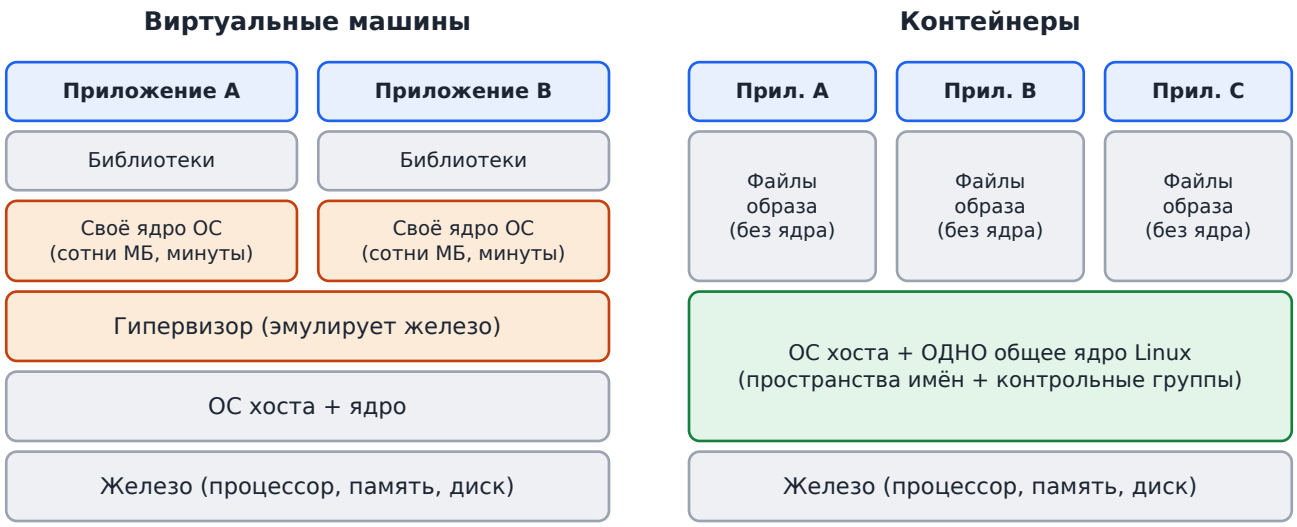


Рис. 1. У виртуальных машин — по ядру на каждую. У контейнеров — одно общее ядро хоста, а в образе лежат только файлы.

Следствия, которые важно запомнить

Факт	Почему так
Контейнер стартует за доли секунды	Не нужно загружать ОС — ядро уже работает, запускается просто процесс.
Образ весит десятки МБ, а не гигабайты	В нём нет ядра, только файлы: библиотеки, программа, утилиты.
На одном сервере можно держать сотни контейнеров	Накладные расходы почти как у обычного процесса.
Linux-контейнеру нужно ядро Linux	Ядро общее, своего у контейнера нет. Поэтому на macOS и Windows Docker Desktop тайно запускает маленькую Linux-ВМ, а контейнеры живут внутри неё.
«Ubuntu» в образе — это не целая Ubuntu	Это только её файлы (apt, bash, библиотеки). Ядро всё равно хоста: `uname -r` внутри контейнера покажет версию ядра хоста.
Изоляция слабее, чем у ВМ	Все контейнеры общаются с одним ядром. Уязвимость в ядре = риск выхода из контейнера. У ВМ граница намного толще.
Процесс контейнера виден на хосте	Запусти контейнер и выполни `ps aux` на хосте — увидишь его процесс в общем списке.

3. Из чего ядро Linux собирает контейнер

Слова «контейнер» в ядре Linux нет. Есть несколько независимых механизмов, и Docker просто включает их все разом для одного процесса. Знать их стоит, потому что каждое странное поведение контейнера объясняется одним из них.

3.1. Пространства имён (namespaces) — «что процесс видит»

Пространство имён подменяет процессу картину мира по какому-то одному ресурсу. Процесс думает, что он один такой, хотя на самом деле живёт среди сотен других.

Пространство	Что изолирует	Что видит процесс в контейнере
pid	Номера процессов	Себя как PID 1, и только своих потомков. Чужие процессы хоста не видны.
net	Сеть	Свои сетевые интерфейсы, свой IP, свои порты, свою таблицу маршрутов. Порт 80 в двух контейнерах не конфликтует.
mnt	Точки монтирования	Свой корень «/» — файлы из образа, а не диск хоста.
uts	Имя машины	Своё имя хоста (обычно короткий ID контейнера).
ipc	Общая память между процессами	Только свои очереди и сегменты общей памяти.
user	Номера пользователей	Может сопоставить «root внутри» с обычным пользователем снаружи. **По умолчанию в Docker выключено!**
cgroup	Видимость контрольных групп	Только своё дерево ограничений.

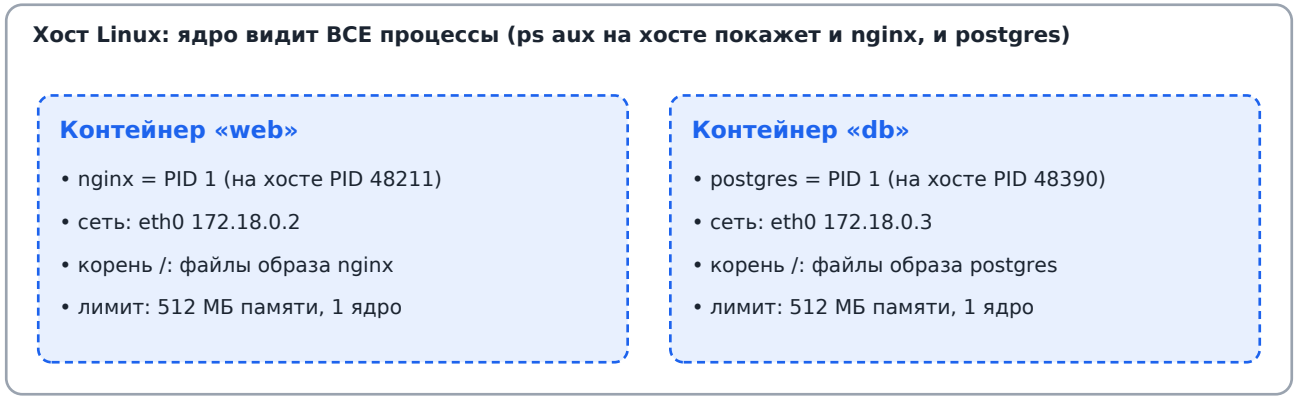


Рис. 2. Внутри каждый думает, что он PID 1 и один в системе. Ядро же видит обычные процессы с обычными номерами.

3.2. Контрольные группы (cgroups) — «сколько процессу можно»

Пространства имён отвечают за видимость, но не за аппетит. Контрольные группы ограничивают ресурсы: память, процессорное время, скорость диска, число процессов. Флаги `--memory=512m` и `--cpus=1.5` — это просто запись чисел в файлы контрольной группы. Если контейнер превысит лимит памяти, ядро убьёт его процесс (так называемый OOM-kill, и в `docker inspect` будет `OOMKilled: true`).

> **Важно**
> Без лимитов контейнер может съесть всю память сервера — изоляция по видимости не означает изоляцию по ресурсам. По умолчанию лимитов нет.

3.3. Слоёная файловая система (overlayfs) — «откуда берутся файлы»

Корень «/» контейнера собирается из нескольких папок, наложенных друг на друга: нижние слои — из образа (только чтение), верхний — пустой слой для записи, свой у каждого контейнера. Когда процесс меняет файл из нижнего слоя, ядро сначала копирует

его наверх и меняет копию («копирование при записи», copy-on-write). Образ при этом не меняется никогда. Подробнее — в разделе 5.

3.4. Урезанные права (capabilities, seccomp, AppArmor/SELinux)

В Linux права root разбиты на ~40 отдельных «способностей» (capabilities): менять сеть, монтировать диски, загружать модули ядра и т.д. Docker по умолчанию отбирает у root внутри контейнера большую часть из них. Плюс фильтр seccomp запрещает около 40+ опасных системных вызовов. Плюс профиль AppArmor или SELinux. Поэтому «root в контейнере» слабее настоящего root — но это всё ещё uid 0 на хосте, если не включено пространство имён user.

> **Итог раздела**
> Контейнер = **процесс** + пространства имён (видимость) + контрольные группы (ресурсы) + слоёная ФС (файлы) + урезанные права (безопасность). Docker — это удобный пульт, который настраивает всё это одной командой `docker run`.

4. Архитектура Docker: кто что делает

Когда ты набираешь `docker run nginx`, работает не одна программа, а цепочка:

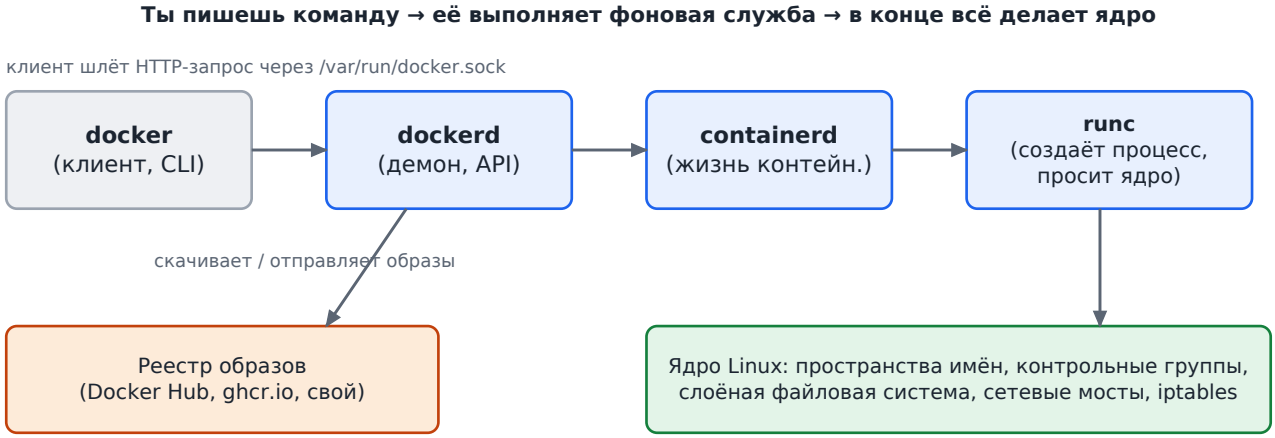


Рис. 3. Цепочка от команды до процесса.

Компонент	Роль
docker (клиент)	Просто программа-пульт. Сама ничего не запускает: превращает команду в HTTP-запрос и шлёт его демону. Может управлять и удалённым демоном (`DOCKER_HOST=ssh://user@server`).
dockerd (демон)	Фоновая служба с правами root. Хранит образы, тома, сети, собирает образы, ходит в реестр. Слушает сокет `/var/run/docker.sock`.
containerd	Управляет жизнью контейнеров: скачанные образы, запуск, остановка. Тот же containerd используется и Kubernetes напрямую, без dockerd.
runc	Маленькая утилита, которая непосредственно просит ядро создать пространства имён, контрольные группы и запускает процесс. Сделала дело — и вышла.
containerd-shim	Прослойка-«родитель» для процесса контейнера. Благодаря ей можно перезапустить dockerd, не убивая работающие контейнеры.

Компонент	Роль
Реестр	Сервер-склад образов (Docker Hub, GitHub Container Registry, свой).

Почему это важно понимать

- Формат образов и контейнеров стандартизирован (стандарт OCI). Образ, собранный Docker, запустит Podman, Kubernetes, containerd — это не «формат Docker», а общий стандарт.
- Доступ к `docker.sock` = полный контроль над демоном, а демон работает от root. Значит, доступ к сокету = root на хосте. К этому вернёмся в разделе 12.
- На macOS/Windows клиент работает у тебя в системе, а демон — внутри скрытой Linux-ВМ. Отсюда странности с производительностью привязанных папок и сетью на этих системах.

5. Образ: слои, имена, теги

Образ (image) — это неизменяемый шаблон: набор слоёв файлов плюс метаданные (какую команду запускать, переменные окружения, рабочая папка, пользователь, открытые порты). Аналогия из программирования: образ — это **класс**, контейнер — **экземпляр** класса. Или: образ — исполняемый файл на диске, контейнер — запущенный процесс.

5.1. Слои

Каждая инструкция Dockerfile, которая меняет файлы (`RUN`, `COPY`, `ADD`), создаёт новый слой — архив с **разницей** относительно предыдущего. Слой идентифицируется хэшем содержимого (sha256), поэтому одинаковые слои хранятся и скачиваются один раз.

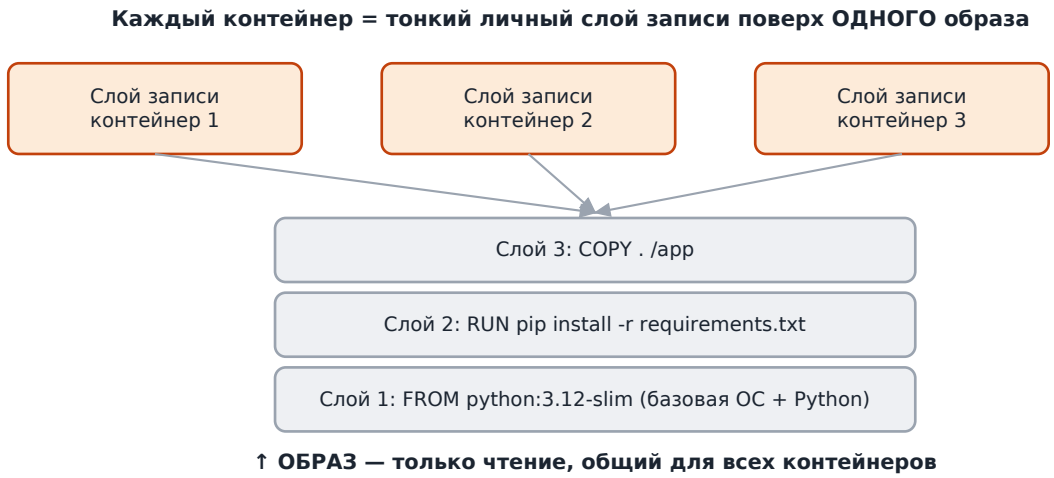


Рис. 4. Образ = стопка слоёв только для чтения. Контейнеры делят его и добавляют сверху по тонкому слою записи.

Следствия:

- 10 контейнеров из одного образа на 500 МБ занимают ~500 МБ, а не 5 ГБ.
- Если два образа построены на `python:3.12-slim`, базовые слои скачаются один раз.
- **Удаление файла в следующем слое не уменьшает образ**: файл остаётся в нижнем слое, сверху просто помечается «удалён». Поэтому скачивание и чистку делают в одном `RUN`.

- Секрет, однажды скопированный в слой, остаётся в образе навсегда, даже если потом его удалить. Любой, кто скачал образ, достанет его командой `docker save`.
- Слой записи контейнера умирает вместе с контейнером. Всё важное — в тома (раздел 8).

5.2. Имя образа

Полное имя образа состоит из частей. Когда ты пишешь просто `nginx`, Docker дописывает недостающее по умолчанию:

```
docker.io / library / nginx : 1.27-alpine
   │          │         │        │
   │          │         │        └─ тег (версия/вариант). Не указан → "latest"
   │          │         └────────── репозиторий (имя образа)
   │          └──────────────────── владелец. "library" = официальные образы
   └─────────────────────────────── адрес реестра. Не указан → Docker Hub

ghcr.io/myname/status-relay:2.1.0       ← свой образ в GitHub Container Registry
nginx@sha256:3b7f...e1a2                 ← по дайджесту: точное, неизменяемое содержимое
```

Понятие	Что значит
Тег	Подвижная метка, как ветка в git. Владелец может перевесить `nginx:1.27` на новую сборку с исправлениями. Один образ может иметь много тегов.
latest	**Не «самая свежая версия»**, а просто тег по умолчанию. Указывает туда, куда его повесил автор. На сервере лучше всегда указывать конкретную версию.
Дайджест	Хэш содержимого (`@sha256:...`). Не меняется никогда. Максимальная воспроизводимость.
Варианты	`-alpine` — на крошечном Alpine Linux (~5 МБ, но своя библиотека musl вместо glibc), `-slim` — урезанный Debian, без суффикса — полный Debian со всеми инструментами.

Образы на машине хранятся у демона в `/var/lib/docker`. Смотреть: `docker images`, историю слоёв: `docker history nginx`, все метаданные: `docker image inspect nginx`. Образы могут быть многоплатформенными (amd64 и arm64 под одним тегом) — Docker сам скачает подходящий под твой процессор.

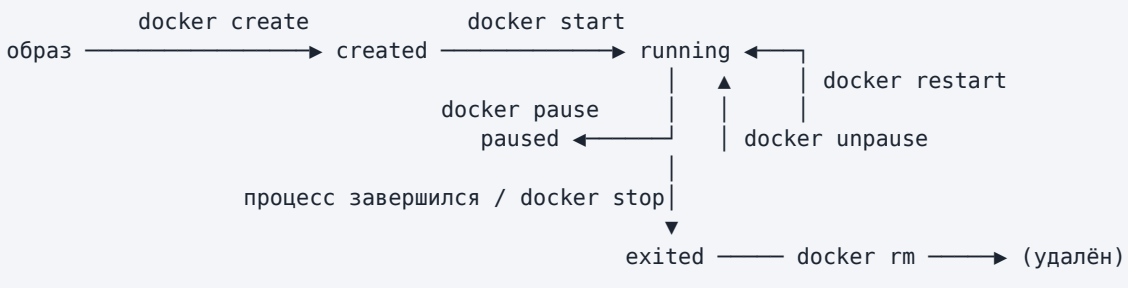
6. Контейнер: жизненный цикл и главный процесс

Контейнер — это образ + слой записи + настройки запуска (имя, порты, тома, переменные, лимиты) + запущенный процесс. Главное правило:

Контейнер живёт, пока жив его главный процесс (PID 1). Процесс завершился — контейнер остановился. Нет никакой «ОС, которая продолжает работать в фоне».

Отсюда частые недоумения новичков: `docker run ubuntu` мгновенно завершается — потому что главный процесс по умолчанию `bash`, а без терминала ему нечего делать, и он выходит. А `docker run -it ubuntu` работает, пока ты не выйдешь из bash.

6.1. Состояния



- **exited** — процесс остановлен, но контейнер существует: его слой записи, логи и настройки на месте. Можно снова `docker start`. Виден в `docker ps -a`.
- Только `docker rm` удаляет контейнер и его слой записи. Флаг `--rm` при запуске — удалить автоматически после остановки (удобно для разовых команд).
- **Код выхода** главного процесса виден в `docker ps -a`: 0 — нормально, 137 — убит сигналом KILL (часто из-за памяти), 143 — завершён сигналом TERM.

6.2. Остановка и сигналы

`docker stop` отправляет главному процессу сигнал SIGTERM («пожалуйста, завершись»), ждёт 10 секунд и, если процесс всё ещё жив, — SIGKILL (убить без разговоров). Сигнал получает только PID 1. Если главный процесс — это оболочка `/bin/sh -c ...`, она часто не передаёт сигнал твоей программе, и та умирает жёстко, не закрыв соединения. Отсюда правило про «exec-форму» в разделе 7.

6.3. Политика перезапуска

Флаг	Поведение
`--restart no`	По умолчанию. Упал — лежит.
`--restart on-failure[:5]`	Перезапускать при ненулевом коде выхода (не более 5 раз).
`--restart always`	Всегда, в том числе при старте демона после перезагрузки сервера.
`--restart unless-stopped`	Как always, но если ты остановил вручную — не поднимать после перезагрузки. Самый популярный вариант для серверов.

6.4. Философия: один контейнер — одна задача

Контейнер не мини-сервер, куда ставят всё подряд. Принято: один главный процесс — одна ответственность. Веб-приложение, база, кэш, обратный прокси — четыре отдельных контейнера, соединённых сетью. Так их проще обновлять, масштабировать, перезапускать и смотреть логи. Контейнеры **одноразовые**: их не чинят и не обновляют изнутри, а пересоздают из нового образа. Всё, что должно пережить пересоздание, хранится снаружи (тома, база).

6.5. Логи

Docker собирает всё, что главный процесс пишет в стандартный вывод (stdout) и вывод ошибок (stderr). Поэтому приложения в контейнерах пишут логи в консоль, а не в файлы. Смотреть: `docker logs -f имя`. По умолчанию логи копятся без ограничения размера — на сервере стоит настроить ротацию (`--log-opt max-size=10m` или в `/etc/docker/daemon.json`).

7. Dockerfile: как собирается образ

Dockerfile — текстовый рецепт образа. Команда `docker build` читает его сверху вниз: берёт базовый образ, на каждом шаге запускает временный контейнер, выполняет инструкцию и сохраняет результат как новый слой.

Dockerfile — пример для веб-приложения на Python

```
# 1. Основа: готовый образ с Python
FROM python:3.12-slim

# 2. Все дальнейшие команды выполняются в /app (папка создастся сама)
WORKDIR /app

# 3. Сначала ТОЛЬКО список зависимостей — ради кэша (см. ниже)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 4. Потом весь остальной код
COPY . .

# 5. Не работать от root
RUN useradd --create-home appuser
USER appuser

# 6. Документация: приложение слушает 8000 (сам порт НЕ открывает)
EXPOSE 8000

# 7. Что запускать при старте контейнера (exec-форма, массив)
CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
docker build -t myapp:1.0 .
#             │          └─ контекст сборки: папка, содержимое которой отправляется демону
#             └────────────── имя и тег будущего образа
```

7.1. Инструкции

Инструкция	Что делает	Слой?
`FROM`	Базовый образ. Всегда первая. `FROM scratch` — абсолютно пустой образ.	—
`WORKDIR`	Рабочая папка для следующих инструкций и для запуска контейнера.	нет
`COPY`	Копирует файлы из контекста сборки в образ.	да
`ADD`	Как COPY, но умеет распаковывать архивы и качать по ссылке. Без нужды не использовать.	да
`RUN`	Выполняет команду во время **сборки** (установка пакетов, компиляция).	да
`ENV`	Переменная окружения — есть и при сборке, и в работающем контейнере.	нет
`ARG`	Переменная только на время сборки (`--build-arg`). В контейнере её нет.	нет
`USER`	От какого пользователя выполнять дальнейшие команды и сам контейнер.	нет
`EXPOSE`	Только пометка «приложение слушает этот порт». Ничего не открывает.	нет
`CMD`	Команда при **запуске** контейнера по умолчанию. Легко заменяется аргументами `docker run образ другая-команда`.	нет

Инструкция	Что делает	Слой?
ENTRYPOINT	Главная программа контейнера. CMD тогда становится её аргументами по умолчанию.	нет
HEALTHCHECK	Команда проверки «жив ли сервис». Статус виден в `docker ps`.	нет
VOLUME	Помечает папку как место для данных (Docker создаст анонимный том).	нет

Ключевое различие: время сборки vs время запуска
`RUN` — выполняется **один раз при сборке**, результат запекается в образ. `CMD`/`ENTRYPOINT` — выполняются **при каждом запуске** контейнера. Путать их — частая ошибка: `RUN python app.py` попытается запустить сервер во время сборки.

7.2. Кэш сборки и порядок инструкций

Docker кэширует каждый слой. При повторной сборке он идёт по инструкциям и, пока ни инструкция, ни входные файлы не изменились, берёт слой из кэша. **Как только один шаг изменился — все шаги ниже пересобираются.** Для `COPY` «изменился» = поменялось содержимое копируемых файлов.

Поэтому в примере сначала копируется только `requirements.txt` и ставятся зависимости, а код — потом. Меняешь код (часто) — пересобирается только последний быстрый слой. Меняешь зависимости (редко) — тогда уже переустановка. Правило: **то, что меняется редко, — выше; то, что часто, — ниже**.

7.3. Контекст сборки и .dockerignore

Точка в `docker build .` — это контекст: вся папка отправляется демону (он может быть на другой машине). `COPY` видит только файлы из контекста. Файл `.dockerignore` (синтаксис как у .gitignore) исключает лишнее: `.git`, `node_modules`, `.env`, логи. Это ускоряет сборку и не даёт случайно запечь секреты в образ.

7.4. CMD и ENTRYPOINT, exec-форма и shell-форма

```
CMD ["nginx", "-g", "daemon off;"]   # exec-форма: процесс nginx сам становится PID 1  ✔
CMD nginx -g "daemon off;"           # shell-форма: PID 1 = /bin/sh -c "...", nginx — его потомок
                                     # → SIGTERM получает sh, а не nginx

ENTRYPOINT ["python", "cli.py"]      # программа фиксирована
CMD ["--help"]                       # аргументы по умолчанию
# docker run img          → python cli.py --help
# docker run img migrate  → python cli.py migrate   (аргументы заменили CMD)
```

7.5. Многоэтапная сборка (multi-stage)

Для компиляции нужны тяжёлые инструменты (компилятор, исходники, кэш), а для работы — только готовый бинарник. Многоэтапная сборка позволяет собрать в одном образе, а в итоговый скопировать только результат. Итоговый образ получается в десятки раз меньше и без лишнего (меньше поверхность атаки).

Dockerfile — сборка в два этапа

```
# Этап 1: сборка (большой образ с компилятором Rust, ~1.5 ГБ)
FROM rust:1.82 AS builder
WORKDIR /src
COPY . .
RUN cargo build --release

# Этап 2: итоговый образ (~80 МБ). Компилятора здесь нет.
FROM debian:bookworm-slim
COPY --from=builder /src/target/release/server /usr/local/bin/server
USER nobody
CMD ["server"]
```

8. Данные: тома и привязанные папки

Слой записи контейнера живёт столько же, сколько контейнер. Пересоздал контейнер (а при обновлении образа это обязательно) — данные пропали. Значит, данные нужно хранить **вне** контейнера и подключать внутрь. Механизм один — монтирование папки в файловую систему контейнера, — а вариантов источника три:

Тип	Откуда берётся	Когда использовать
Том (volume)	Папка, которой управляет Docker: `/var/lib/docker/volumes/имя/_data`. Создаётся и живёт отдельно от контейнеров.	Данные баз и сервисов на сервере. Основной способ хранения.
Привязанная папка (bind mount)	Любая конкретная папка хоста по абсолютному пути.	Разработка (код с хоста виден в контейнере сразу), конфиги, сертификаты.
tmpfs	Оперативная память. Исчезает при остановке.	Временные и чувствительные файлы, которые не должны попасть на диск.

```
# Том: Docker создаст том pgdata, если его нет
docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16

# Привязанная папка: путь начинается с / (или ./ в compose) — значит, папка хоста
docker run -d -v /srv/nginx/conf.d:/etc/nginx/conf.d:ro nginx
#                                                   └─ ro = только чтение внутри

# Синтаксис -v ИСТОЧНИК:КУДА_В_КОНТЕЙНЕРЕ[:опции]
# Более явный вариант того же самого:
docker run --mount type=volume,src=pgdata,dst=/var/lib/postgresql/data postgres:16
```

Тонкости, которые объясняют «странности»

- Монтирование **перекрывает** то, что лежало в этой папке образа. Привязал пустую папку хоста на `/app` — код из образа «исчез» (он на месте, просто закрыт сверху).
- Исключение: **пустой новый том** при первом подключении заполняется содержимым образа из этой папки. Привязанная папка — никогда.
- Тома переживают `docker rm`. Удаляются только явно: `docker volume rm`, `docker compose down -v`, `docker volume prune`. Это защита от случайной потери данных.
- **Права доступа**: внутри контейнера номера пользователей те же, что на хосте. Если процесс работает как uid 1000, а папка хоста принадлежит root — будет «Permission denied».

- Бэкап тома — это бэкап его папки, но для баз данных правильнее делать дамп средствами самой базы (`pg_dump`), а не копировать файлы на ходу.

9. Сети и порты

Благодаря пространству имён net у каждого контейнера своя сеть: свой интерфейс `eth0`, свой IP, свой `localhost`. Docker соединяет эти изолированные сети между собой и с внешним миром с помощью виртуальных мостов и правил iptables.

9.1. Типы сетей (драйверы)

Драйвер	Как работает	Когда
bridge	Виртуальный коммутатор на хосте. Контейнеры получают внутренние IP (172.17.x.x и т.п.), наружу ходят через NAT хоста.	По умолчанию. 95% случаев.
host	Нет отдельной сети: контейнер использует сеть хоста напрямую. Порты не пробрасываются — приложение сразу слушает на хосте.	Максимальная сетевая скорость, особые сетевые утилиты. Изоляции сети нет.
none	Только `lo`. Никакой сети.	Пакетная обработка, где сеть не нужна.
overlay	Единая сеть поверх нескольких серверов.	Swarm, кластеры.
macvlan	Контейнер выглядит в локальной сети как отдельное устройство со своим MAC.	Редкие специфичные случаи.

9.2. Сеть по умолчанию против своей сети

Если не указать сеть, контейнер попадёт в стандартную сеть `bridge` (мост `docker0`). У неё есть важный недостаток: **контейнеры не находят друг друга по имени**, только по IP, а IP меняется при пересоздании.

В **своей** (пользовательской) сети работает встроенный DNS Docker (адрес `127.0.0.11` внутри контейнера): имя контейнера или сервиса превращается в его текущий IP. Поэтому для связки контейнеров всегда создают свою сеть. Compose делает это автоматически.

```
docker network create app_net
docker run -d --name db  --network app_net -e POSTGRES_PASSWORD=secret postgres:16
docker run -d --name web --network app_net -p 8080:80 my-web
# внутри web строка подключения: postgres://postgres:secret@db:5432/postgres
#                                                          └─ имя контейнера = адрес
```

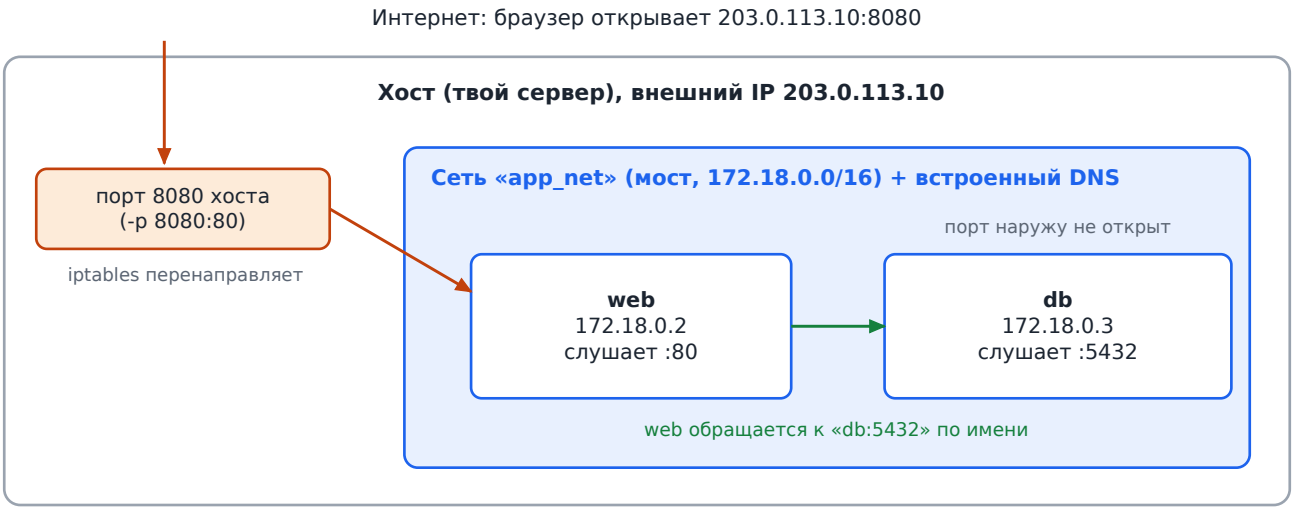


Рис. 5. Наружу открыт только порт web. База доступна лишь внутри сети app_net — по имени «db».

9.3. Проброс портов

Сеть контейнера изолирована, поэтому снаружи до него не достучаться, пока не пробросить порт: `-p ПОРТ_ХОСТА:ПОРТ_КОНТЕЙНЕРА`. Docker добавляет правило iptables: всё, что пришло на порт хоста, перенаправить на IP контейнера.

Запись	Смысл
`-p 8080:80`	Порт 8080 **на всех интерфейсах хоста** (0.0.0.0) → порт 80 контейнера. Доступно из интернета.
`-p 127.0.0.1:8080:80`	Только с самого хоста. Типично, когда спереди стоит nginx хоста или доступ через SSH-туннель.
`-p 53:53/udp`	Проброс UDP (по умолчанию — TCP).
`-P`	Пробросить все порты из EXPOSE на случайные свободные порты хоста.

Ловушка: проброшенный порт обходит ufw
Docker пишет свои правила iptables **в обход** обычной цепочки входящих правил, поэтому `ufw deny 8080` не закроет порт, проброшенный через `-p 8080:80`. Решение: пробрасывать только на `127.0.0.1`, не пробрасывать то, что не должно быть публичным (базы!), или настраивать цепочку `DOCKER-USER`.

Ловушка: что такое localhost
Внутри контейнера `localhost` — это **сам контейнер**, а не хост и не соседний контейнер. Приложение в контейнере web не достучится до базы по `localhost:5432` — нужно имя `db`. И наоборот: сервер внутри контейнера должен слушать `0.0.0.0`, а не `127.0.0.1`, иначе проброшенный порт будет вести в пустоту. Чтобы из контейнера попасть на хост, используют `host.docker.internal` (на Linux — с флагом `--add-host=host.docker.internal:host-gateway`).

10. Реестры: где живут образы

Реестр — сервер, хранящий образы, как git-хостинг хранит репозитории. Образ собирают один раз, отправляют в реестр, а на любом сервере скачивают.

Реестр	Заметки
Docker Hub (`docker.io`)	По умолчанию. Официальные образы (nginx, postgres, redis...). Есть ограничение на число скачиваний для анонимных пользователей.
GitHub Container Registry (`ghcr.io`)	Удобно, если код на GitHub: собирать и публиковать через GitHub Actions.
GitLab, облачные реестры (AWS ECR, Google AR и др.)	Привязаны к своим платформам.
Свой реестр	Образ `registry:2` или Harbor — полный контроль, приватность.

```
docker login ghcr.io                          # авторизация (токен вместо пароля)
docker tag myapp:1.0 ghcr.io/me/myapp:1.0     # дать образу имя с адресом реестра
docker push ghcr.io/me/myapp:1.0              # отправить (уйдут только недостающие слои)
docker pull ghcr.io/me/myapp:1.0              # скачать на сервере
```

`docker tag` не копирует образ — он вешает ещё одну метку на тот же образ (как ещё одна ссылка на тот же объект). Отправка идёт по слоям: если слои уже есть в реестре, они не передаются повторно. Скачивая чужие образы, отдавай предпочтение официальным (`library/`) и проверенным издателям: образ — это чужой код, который ты запускаешь у себя.

11. Docker Compose: много контейнеров одним файлом

Реальное приложение — это несколько контейнеров: сервис, база, кэш, прокси. Запускать их длинными командами `docker run` с десятком флагов неудобно и нельзя положить в git. **Compose** позволяет описать всё в одном YAML-файле и управлять как единым целым.

Ментальная модель
Compose — **не новая технология**. Он просто читает файл и выполняет то же самое, что ты сделал бы руками: создаёт сеть, тома, собирает образы и вызывает аналог `docker run` для каждого сервиса. Каждый ключ в файле соответствует флагу `docker run`.

compose.yaml

```
services:                       # каждый сервис = один (или несколько одинаковых) контейнер
  web:
    build: .                    # собрать образ из Dockerfile в этой папке
    image: myapp:1.0            # ...и назвать его так
    ports:
      - "127.0.0.1:8000:8000"   # = -p
    environment:                # = -e
      DATABASE_URL: postgres://app:${DB_PASSWORD}@db:5432/app
    depends_on:
      db:
        condition: service_healthy   # ждать, пока db пройдёт проверку здоровья
    restart: unless-stopped     # = --restart

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${DB_PASSWORD}   # подставится из файла .env рядом
      POSTGRES_DB: app
    volumes:
      - pgdata:/var/lib/postgresql/data   # именованный том
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
      retries: 10
    restart: unless-stopped
    # ports не указаны → база не торчит наружу, доступна только сервису web

volumes:
  pgdata:                       # объявление тома (создастся при первом up)
```

11.1. Что Compose делает автоматически

- **Проект.** Имя проекта = имя папки (или `-p имя`). Все объекты получают префикс: контейнер `myproj-web-1`, том `myproj_pgdata`.
- **Сеть.** Создаётся сеть `myproj_default`, все сервисы в ней, и каждый доступен соседям **по имени сервиса** (`db`, `web`). Вот почему в адресе базы стоит `@db:5432`.
- **Переменные.** `${DB_PASSWORD}` подставляется из окружения оболочки или из файла `.env` рядом с compose.yaml. `.env` — в .gitignore.
- **Пересоздание по изменениям.** `docker compose up -d` сравнивает файл с текущим состоянием и пересоздаёт только те контейнеры, чья конфигурация или образ поменялись.

11.2. depends_on: тонкость

Просто `depends_on: [db]` гарантирует лишь **порядок запуска**: сначала стартует контейнер db, потом web. Но «контейнер запущен» не значит «Postgres готов принимать соединения» — на это нужны секунды. Поэтому используют `condition: service_healthy` вместе с `healthcheck`, как в примере, а приложение всё равно учат переподключаться.

11.3. Команды Compose

Команда	Что делает
`docker compose up -d`	Создать/обновить и запустить всё в фоне. Главная команда.
`docker compose up -d --build`	То же, но сначала пересобрать образы сервисов с `build:`.

Команда	Что делает
`docker compose down`	Остановить и удалить контейнеры и сеть. **Тома остаются.**
`docker compose down -v`	То же + удалить тома. **Данные базы будут уничтожены.**
`docker compose ps`	Состояние контейнеров проекта.
`docker compose logs -f web`	Логи сервиса в реальном времени (без имени — всех).
`docker compose exec web sh`	Зайти внутрь работающего контейнера сервиса.
`docker compose run --rm web cmd`	Разовый контейнер сервиса для команды (миграции и т.п.).
`docker compose pull`	Скачать свежие версии образов.
`docker compose restart web`	Перезапустить (без применения изменений файла!).
`docker compose config`	Показать итоговый файл со всеми подстановками — для проверки.

Если встретишь старые инструкции
Раньше была отдельная программа `docker-compose` (через дефис) и строка `version: "3.8"` в начале файла. Сейчас Compose встроен: `docker compose` (через пробел), файл предпочтительно называется `compose.yaml`, а поле `version` устарело и игнорируется.

12. Безопасность: что важно знать

Из ментальной модели «контейнер = процесс с общим ядром» прямо вытекают главные риски.

Риск	Почему	Что делать
Группа docker = root	Кто может говорить с демоном, тот может запустить `docker run -v /:/host ...` и получить весь диск хоста.	Давать доступ к docker только тем, кому дал бы root. Рассмотреть rootless-режим или Podman.
Сокет внутри контейнера	Проброс `/var/run/docker.sock` в контейнер (популярно у панелей и Traefik) = root на хосте для этого контейнера.	Только при реальной необходимости, лучше через прокси сокета с ограниченными правами.
root в контейнере	Без пространства имён user это uid 0 хоста, хоть и с урезанными правами. При побеге из контейнера — root.	`USER` в Dockerfile, `--user`, или включить переназначение пользователей (userns-remap).
`--privileged`	Снимает почти все ограничения: все права, доступ к устройствам. Фактически отключает изоляцию.	Не использовать без понимания. Если нужно одно право — `--cap-add` именно его.
Общее ядро	Уязвимость ядра затрагивает все контейнеры сразу.	Обновлять ядро хоста. Для недоверенного кода — ВМ или gVisor/Kata.
Секреты в образе	Всё, что попало в слой (`COPY .env`, `ARG TOKEN`), остаётся в образе и истории.	Секреты передавать при запуске (переменные, файлы, секреты Compose), `.dockerignore`.
Открытые порты	`-p` слушает 0.0.0.0 и обходит ufw.	Пробрасывать на `127.0.0.1`, базы наружу не открывать.

Риск	Почему	Что делать
Чужие образы	Образ = чужой код + чужие уязвимые библиотеки.	Официальные образы, конкретные версии, сканирование (`docker scout`, Trivy).

```
# Пример «укреплённого» запуска
docker run -d --name api \
  --user 1000:1000 \
  --read-only --tmpfs /tmp \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --memory 256m --cpus 0.5 --pids-limit 200 \
  -p 127.0.0.1:8000:8000 \
  myapp:1.0

# --user                 не root
# --read-only --tmpfs    корень только для чтения, писать можно лишь в /tmp
# --cap-drop ALL         отобрать все особые права root
# no-new-privileges      запретить повышение прав через setuid-программы
# --pids-limit           защита от «бомбы» из бесконечных процессов
```

13. Шпаргалка команд

Логика команд: `docker ОБЪЕКТ ДЕЙСТВИЕ` (`docker container ls`, `docker image rm`, `docker volume create`). У самых частых есть короткие старые синонимы: `docker ps` = `docker container ls`, `docker rmi` = `docker image rm`. Справка по любой: `docker КОМАНДА --help`.

Запуск контейнера: флаги docker run

Флаг	Смысл
`-d`	В фоне (detached). Без него терминал привязан к выводу контейнера.
`-it`	Интерактивно с терминалом (для оболочек: `docker run -it ubuntu bash`).
`--name web`	Своё имя вместо случайного.
`--rm`	Удалить контейнер после остановки.
`-p 8080:80`	Проброс порта хост:контейнер.
`-v том:/путь`	Подключить том или папку хоста.
`-e КЛЮЧ=знач`	Переменная окружения. `--env-file .env` — из файла.
`--network сеть`	Подключить к сети.
`--restart unless-stopped`	Политика перезапуска.
`--memory 512m --cpus 1`	Лимиты ресурсов.
`-w /app, -u 1000`	Рабочая папка, пользователь.
`образ команда арг`	Всё после имени образа заменяет CMD.

Контейнеры

Команда	Что делает
`docker ps`	Работающие контейнеры. `-a` — все, включая остановленные.
`docker stop / start / restart имя`	Остановить (TERM → KILL через 10 с) / запустить / перезапустить.
`docker kill имя`	Убить сразу (KILL).
`docker rm имя`	Удалить остановленный. `-f` — даже работающий.
`docker logs -f --tail 100 имя`	Логи: следить, последние 100 строк.
`docker exec -it имя sh`	Запустить ещё один процесс (оболочку) в работающем контейнере.
`docker inspect имя`	Всё о контейнере в JSON: IP, тома, переменные, код выхода.
`docker stats`	Потребление процессора и памяти в реальном времени.
`docker top имя`	Процессы внутри контейнера.
`docker cp имя:/путь ./куда`	Скопировать файлы из контейнера (или обратно).

Команда	Что делает
`docker port имя`	Какие порты проброшены.

Образы

Команда	Что делает
`docker images`	Список образов.
`docker pull образ:тег`	Скачать.
`docker build -t имя:тег .`	Собрать. `-f путь/Dockerfile` — другой файл, `--no-cache` — без кэша.
`docker tag src:1 dst:1`	Дать образу ещё одно имя.
`docker push имя:тег`	Отправить в реестр.
`docker rmi образ`	Удалить образ.
`docker history образ`	Слои и их размеры (видно, что раздуло образ).
`docker save / load`	Образ в tar-архив и обратно (перенос без реестра).

Тома и сети

Команда	Что делает
`docker volume ls / create / rm / inspect`	Управление томами. inspect покажет, где том лежит на диске.
`docker network ls / create / rm / inspect`	Управление сетями. inspect покажет подключённые контейнеры и их IP.
`docker network connect сеть имя`	Подключить работающий контейнер к ещё одной сети.

Очистка и диагностика

Команда	Что делает
`docker system df`	Сколько места занимают образы, контейнеры, тома, кэш сборки.
`docker container prune`	Удалить все остановленные контейнеры.
`docker image prune`	Удалить «висячие» образы (без тега). `-a` — все неиспользуемые.
`docker builder prune`	Очистить кэш сборки (часто занимает больше всего).
`docker volume prune`	Удалить тома, не подключённые ни к одному контейнеру. **Осторожно с данными.**
`docker system prune`	Всё неиспользуемое разом (кроме томов; с `--volumes` — и их).
`docker info / version`	Состояние демона, драйвер хранения, версии.
`docker events`	Поток событий демона в реальном времени.

Типичный порядок отладки

1. `docker ps -a` — контейнер вообще работает? Какой код выхода?
2. `docker logs имя` — что сказал процесс перед смертью?
3. `docker inspect имя` — правильные ли переменные, тома, сеть, команда запуска?
4. `docker exec -it имя sh` — зайти и посмотреть изнутри: файлы, `env`, доступность соседей.
5. Не запускается совсем? `docker run -it --rm --entrypoint sh образ` — открыть оболочку вместо обычной команды и проверить руками.

14. Итоговая картина

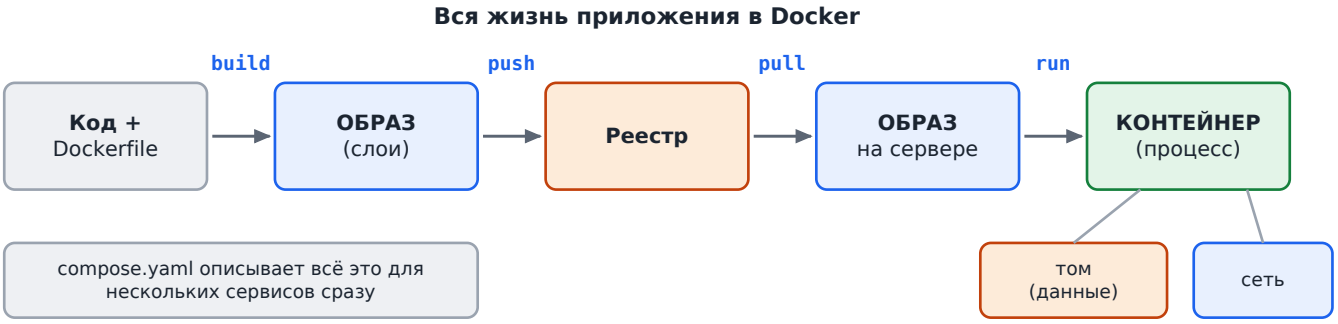


Рис. 6. Путь от кода до работающего сервиса.

Вся модель в десяти утверждениях

1. **Контейнер** — обычный процесс Linux, которому ядро ограничило видимость (пространства имён) и ресурсы (контрольные группы). Ядро общее с хостом.
2. **Образ** — неизменяемая стопка слоёв файлов + метаданные запуска. Образ относится к контейнеру как класс к экземпляру.
3. **Dockerfile** — рецепт образа. Каждый шаг, меняющий файлы, — слой. Слои кэшируются, порядок шагов решает скорость сборки.
4. `RUN` работает при сборке, `CMD`/`ENTRYPOINT` — при каждом запуске.
5. Контейнер живёт, пока жив его PID 1. Логи — это его stdout/stderr.
6. Контейнеры одноразовые: их не чинят, а пересоздают из нового образа. Слой записи умирает вместе с контейнером.
7. Всё ценное — в **томах** (или привязанных папках), которые живут отдельно от контейнеров.
8. У каждого контейнера своя сеть и свой `localhost`. В своей сети контейнеры находят друг друга по имени; снаружи доступно только то, что проброшено через `-p`.
9. **Compose** — тот же `docker run`, только описанный файлом для нескольких сервисов, с общей сетью и томами.
10. Демон работает от root: доступ к Docker = root на хосте; изоляция контейнера тоньше, чем у ВМ.

Как это сопоставить: команда → что происходит на самом деле

Ты делаешь	Что происходит внутри
`docker run -d -p 8080:80 -v data:/data --memory 256m nginx`	Клиент → API демона. Демон проверяет образ (нет — скачивает слои из реестра), создаёт том data, собирает из слоёв корень ФС через overlayfs, добавляет слой записи, монтирует том в /data. containerd + runc создают пространства имён, контрольную группу с лимитом 256 МБ, виртуальный интерфейс в мосту, правило iptables для 8080 → 80 и запускают процесс nginx как PID 1.
`docker stop`	SIGTERM процессу, через 10 с — SIGKILL. Слой записи остаётся.
`docker rm`	Удаляются слой записи и настройки. Том data — остаётся.

Ты делаешь	Что происходит внутри
`docker build`	Временные контейнеры для каждого шага Dockerfile, результат каждого шага — слой.

Куда углубляться дальше (по мере необходимости)

Направление	Что там
Linux под капотом	Создать контейнер без Docker руками: `unshare`, `chroot`, файлы в `/sys/fs/cgroup`. Лучший способ закрепить раздел 3.
Хорошие образы	Многоэтапная сборка, минимальные базы (distroless, scratch, alpine), кэш-монтирования BuildKit, сборка под несколько архитектур (`docker buildx`).
Безопасность	Rootless Docker, Podman, переназначение пользователей, seccomp-профили, сканирование образов, подписи образов.
Доставка	Сборка и публикация образов в CI (GitHub Actions → ghcr.io), автообновление на сервере.
Обратный прокси	Traefik / Caddy / nginx перед контейнерами: домены, HTTPS, маршрутизация.
Оркестрация	Kubernetes: те же образы, но много серверов, самовосстановление, масштабирование.

Последний совет
Когда что-то в Docker ведёт себя непонятно, задай себе три вопроса: **какой процесс здесь PID 1? какие файлы он видит** (образ + тома)? **какая у него сеть**? Почти любая проблема отвечает на один из них.